

■ The 'this' Pointer in C++

1■■ What is an Object in C++?

When you create an object of a class, memory is allocated for that object's data members.

```
class Person { string name; int age; public: void show() { cout <<
"Name: " << name << ", Age: " << age << endl; } }; Person p1, p2; // p1
and p2 are two different objects
```

Each object has its own copy of **name** and **age**.

2■■ The Hidden 'this' Pointer

When a non-static member function is called, C++ automatically passes the address of the object as a hidden argument using a pointer named **this**.

```
p1.show(); // becomes internally: show(&p1); p2.show(); // becomes
internally: show(&p2);
```

this == &p1; when p1.show() is called.

this == &p2; when p2.show() is called.

3■■ Practical Example

```
class Person { string name; int age; public: Person(string n, int a) {
name = n; age = a; } void display() { cout << "Name: " << this->name <<
", Age: " << this->age << endl; } }; Person p1("Khalid", 18); Person
p2("Danish", 20); p1.display(); p2.display();
```

■ When **p1.display()** runs, **this** points to **p1**.

■ When **p2.display()** runs, **this** points to **p2**.

4■■ Why is 'this' Useful?

(a) Resolving Naming Conflicts

```
void setName(string name) { this->name = name; // left: data member,
right: parameter }
```

(b) Returning the Current Object

```
Person& older(Person &x;) { if (x.age > this->age) return x; else return
*this; }
```

(c) Method Chaining

```
Person& setAge(int a) { this->age = a; return *this; }
p1.setAge(20).display();
```

■ Summary

Concept	Meaning
this	Pointer automatically passed to member functions
*this	Actual object pointed to by 'this'
Purpose	Identifies which object is calling the function
Used in	Resolving naming conflicts, returning current object, method chaining